

The Verified-Candidate Loop for Budgeted Large Language Model-Assisted High-Level Synthesis

Team NULL (Track A, Team 46474)

Abstract—High-level synthesis (HLS) agents must repair and optimize unfamiliar kernels under limited model and tool budgets. The Verified-Candidate Loop (VCL) promotes a candidate only after tool-based validation, using a bounded quality-of-results (QoR) score over effective latency, capacity-normalized resources, and source-change evidence. QoR-guided retrieval reuses compatible evidence, while Failure Reflection converts diagnostics into constraints without extra model calls. We evaluate three open-weight large language model endpoints on a balanced 150-task suite, then independently recheck every public-completed kernel with hidden tests and reference synthesis. All 432/432 evaluated kernels pass evaluator-side hidden correctness. On the fully scoreable 25-task optimization slice, DeepSeek V4 Pro leads the starter-anchored proxy (85.5) and reference-anchored score (83.11). Qwen3.6-27B has the highest public-gate completion (98.7%, 148/ 150), and Qwen3.5-122B-A10B uses the fewest tokens. Because each endpoint has one campaign, these results compare endpoints rather than architectures.

Index Terms—high-level synthesis, LLM agent, FPGA, retrieval-augmented generation, quality of results

I. GATE EVIDENCE, NOT PROMPTS

LLM agents can edit unfamiliar HLS kernels, but only tool reports certify correctness and hardware quality. A metered agent therefore needs a promotion rule that preserves a verified fallback when any required validation gate fails [1].

Prior HLS agents automate code transformation, directive search, and feedback-driven repair [2]–[4]. Tool-using language agents alternate reasoning and actions [5]. Our design assigns candidate promotion to deterministic HLS evidence under explicit model and tool budgets.

The Verified-Candidate Loop (VCL) contributes three findings. **(1) Validation contract.** A fail-closed workflow preserves a verified fallback across seven gates: Interface, CSIM, SYNTH, Frequency, Capacity, required CoSIM, and Metric completeness. **(2) Hardware objective.** The bounded score Q_{HW} combines effective latency, capacity-normalized resource footprint, and source-change evidence to reject costly speedups. **(3) Independent endpoint evaluation.** Hidden reevaluation checks every public-completed kernel; on the fully scoreable 25-task optimization slice, DeepSeek leads both QoR measures, while Qwen3.6 leads public completion and Qwen3.5 minimizes token use (Section IV).

II. TOOL EVIDENCE CONTROLS THE AGENT WORKFLOW

The task interface exposes an editable kernel, public tests, target part, and budgets while withholding hidden tests and frozen references. The controller records every candidate, gate, metric, token, and credit in one run state.

The VCL runs seven gates in order: Interface, CSIM, SYNTH, Frequency, Capacity, required CoSIM, and Metric completeness. Frequency enforces 100 MHz, Capacity enforces U55C limits, and CoSIM applies only when required. Define validity $V(c) = \prod_i G_i$ over applicable gates $G_i \in \{0, 1\}$; an inapplicable gate contributes 1. Let b be the verified fallback. The promotion rule is $P(c) = V(c) \wedge [Q_{HW}(c) > Q_{HW}(b)]$.

Gate failures fall into three actionable classes, each driving a specific repair strategy without extra LLM calls. **Interface mismatches** (112 of 691 candidates, 16%) trigger API-usage constraints fixing function signatures and pragma syntax. **Synthesis bottlenecks** include timing violations, resource overruns, and II blow-up. QoR-RAG converts them into directive constraints such as pipeline, unroll, and array partitioning. The controller repairs **co-simulation deadlocks** via handshake insertion and port-width correction. Duplicate candidates and repeated failures stop retries.

Figure 1 summarises the loop. Across three campaigns: 691 candidates, 112 interface rejections, 487 synthesis gates, 67 CoSIM gates. Co-simulation failures incur the highest per-repair token cost, consistent with deadlock complexity.

III. ONE SCORE REJECTS FALSE SPEEDUPS

Q_{HW} combines performance gain and resource cost into a single bounded utility. Let anchor a and candidate c have cycle latency L , effective period p , and initiation interval II . The performance ratio $r_P = (p_a L_a)/(p_c L_c)$ captures speedup. The area ratio r_A uses a capacity-normalized footprint $F(x) = \sum_r x_r / K_r$ over LUT, FF, DSP, BRAM, and URAM to prevent discontinuous jumps from resource transfers. Two evidence factors reward source edits (D , 1.01 $\times$) and validity repair (F , 2 $\times$). The aggregate hardware ratio $h = 1.01^D 2^F r_P^{0.55} r_A^{0.45}$ weights performance at 55% and area at 45%, calibrated against 99 frozen PPA references.

$Q_{HW} = 1 - 1/(1 + h)^2$ maps $h=1$ (no change) to 0.75, $h \rightarrow \infty$ to 1, and $h \rightarrow 0$ to 0. The mapping degrades continuously for bloated speedups. Budget efficiency $E = \max(0.80, 1 - 0.10u_{\text{credit}} - 0.10u_{\text{time}})$ discounts excessive tool calls with a floor at 0.80, where each u is the consumed fraction clipped to $[0, 1]$. The final score is

$$S = 100 V(c) Q_{HW} E. \quad (1)$$

Promotion compares Q_{HW} only after $V(c)=1$; a failed gate yields $S=0$, and the efficiency floor guarantees a valid baseline at least 60 points.

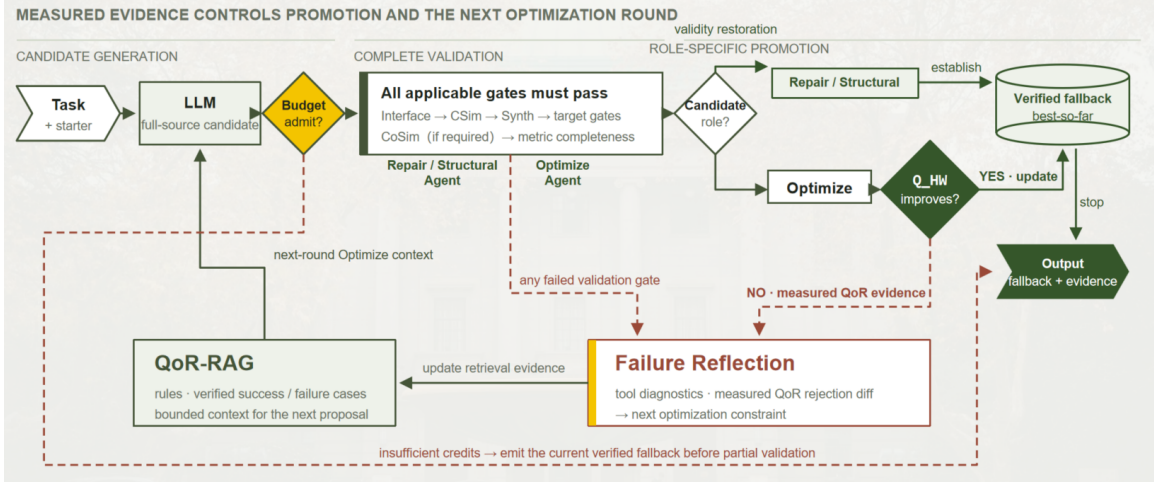


Fig. 1. **The Verified-Candidate Loop.** A candidate replaces the verified fallback only after every applicable gate passes and Q_{HW} improves.

IV. INDEPENDENT CHECKS SEPARATE THREE OBJECTIVES

Protocol. Three hosted endpoints process the same balanced 150-task suite under one agent revision and the Track-A contract [6]. Submission-side synthesis provides public-gate completion and a starter-anchored QoR proxy. A fresh evaluator reruns all 432 public-completed kernels with hidden CSim, synthesis, and required hidden CoSim, then computes reference-anchored QoR when fixed reference metrics exist. Both QoR measures apply Eq. (1); the 25 optimization tasks (QoR-25) have complete coverage.

TABLE I
INDEPENDENT EVALUATION OF PUBLIC-COMPLETED KERNELS. HIDDEN USES PUBLIC AS ITS DENOMINATOR; PROXY AND REF. QoR ARE STARTER- AND REFERENCE-ANCHORED MEANS ON THE FULLY COVERED QoR-25 SLICE.

Endpoint	Public	Hidden	Tokens (M)	Proxy	Ref. QoR
DeepSeek V4 Pro	144/150	144/144	5.87	85.5	83.11
Qwen3.5-122B-A10B	140/150	140/140	1.68	76.3	70.77
Qwen3.6-27B	148/150	148/148	1.92	79.7	75.33

Results. Table I yields three patterns.

(1) **No hidden failures appear.** All 432/432 kernels pass evaluator-side hidden correctness. Qwen3.6-27B supplies the largest evaluated set (148/150 public completions), including all 25 structural CoSim tasks.

(2) **DeepSeek leads both QoR-25 measures.** The shared denominator makes the endpoints directly comparable, and changing from the starter to the reference anchor preserves their ordering: DeepSeek, Qwen3.6, then Qwen3.5.

(3) **No endpoint dominates every objective.** Qwen3.6 leads public-gate completion, DeepSeek leads QoR-25, and Qwen3.5 minimizes token use. One campaign per endpoint prevents architecture attribution.

V. LIMITS AND NEXT STEPS

The VCL makes validation evidence and Q_{HW} the promotion authority. Independent reevaluation supports its functional-correctness claims: all 432 public-completed kernels pass hidden checks. QoR-25 also preserves the endpoint ordering across starter and reference anchors, with DeepSeek leading both measures; Qwen3.6 leads public completion, and Qwen3.5 minimizes token use.

A full-corpus reference ranking remains unwarranted because fixed anchor metrics are available for only 109–115 of 150 tasks. We therefore treat the fully covered QoR-25 slice as the primary quality comparison. Evidence comes from one Vitis 2025.2/U55C campaign per endpoint; matched repeats, ablations, and broader platforms remain future work.

REFERENCES

- [1] Advanced Micro Devices, Inc., *Vitis High-Level Synthesis User Guide*, AMD, 2026, version 2025.2. [Online]. Available: <https://docs.amd.com/r/2025.2-English/ug1399-vitis-hls>
- [2] C. Xiong, C. Liu, H. Li, and X. Li, “HLSPilot: LLM-based high-level synthesis,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [3] H. Xu, H. Hu, and S. Huang, “Optimizing high-level synthesis designs with retrieval-augmented large language models,” in *2024 IEEE LLM Aided Design Workshop*, 2024, pp. 1–5.
- [4] R. Li, J. Xiong, X. He, J. Zhao, J. Lv, H. Fang, L. Qi, and X. Wang, “ChatHLS: Towards systematic design automation and optimization for high-level synthesis,” *arXiv preprint arXiv:2507.00642*, 2026. [Online]. Available: <https://arxiv.org/abs/2507.00642>
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [6] FPT 2026 Organizing Committee, “FPT’26 Design Competition: Track A,” <https://fpt2026.uark.edu/fpt26-design-competition/>, 2026, accessed July 2026.